DEADLOCK

MD. NASERULLAH KAZI ZILANI MD. RAJIBUL HASAN

INTRODUCTION

What Is Deadlock

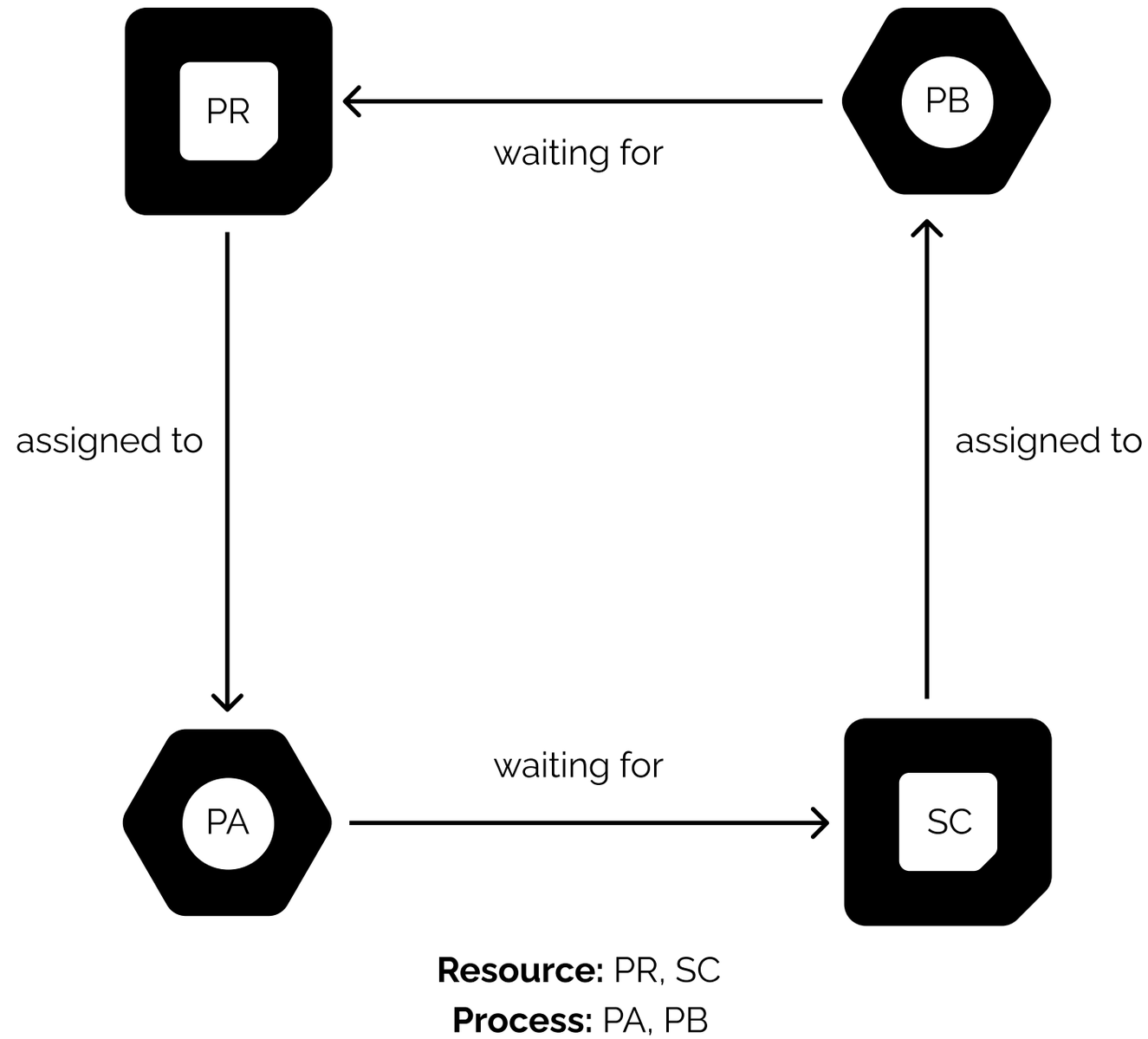
A deadlock in an Operating System is a situation where two or more processes get stuck forever because each is waiting for a resource held by another.

Deadlock Scenario

Imagine this:

- Process A has Printer
- Process B has Scanner
- Process A now wants Scanner
- Process B now wants Printer

Now both wait forever. Nobody can continue.
That's deadlock.



Deadlock Conditions

For a deadlock to occur, four necessary conditions (known as the Coffman conditions) must hold simultaneously, If even one condition is prevented, deadlock cannot occur.

Mutual Exclusion

A resource can be used by only one process at a time.

> Example: one printer cannot print two jobs at the exact same instant.

Circular Wait

There is a cycle of waiting:

- P1 waits for P2
- P2 waits for P3
- P3 waits for P1

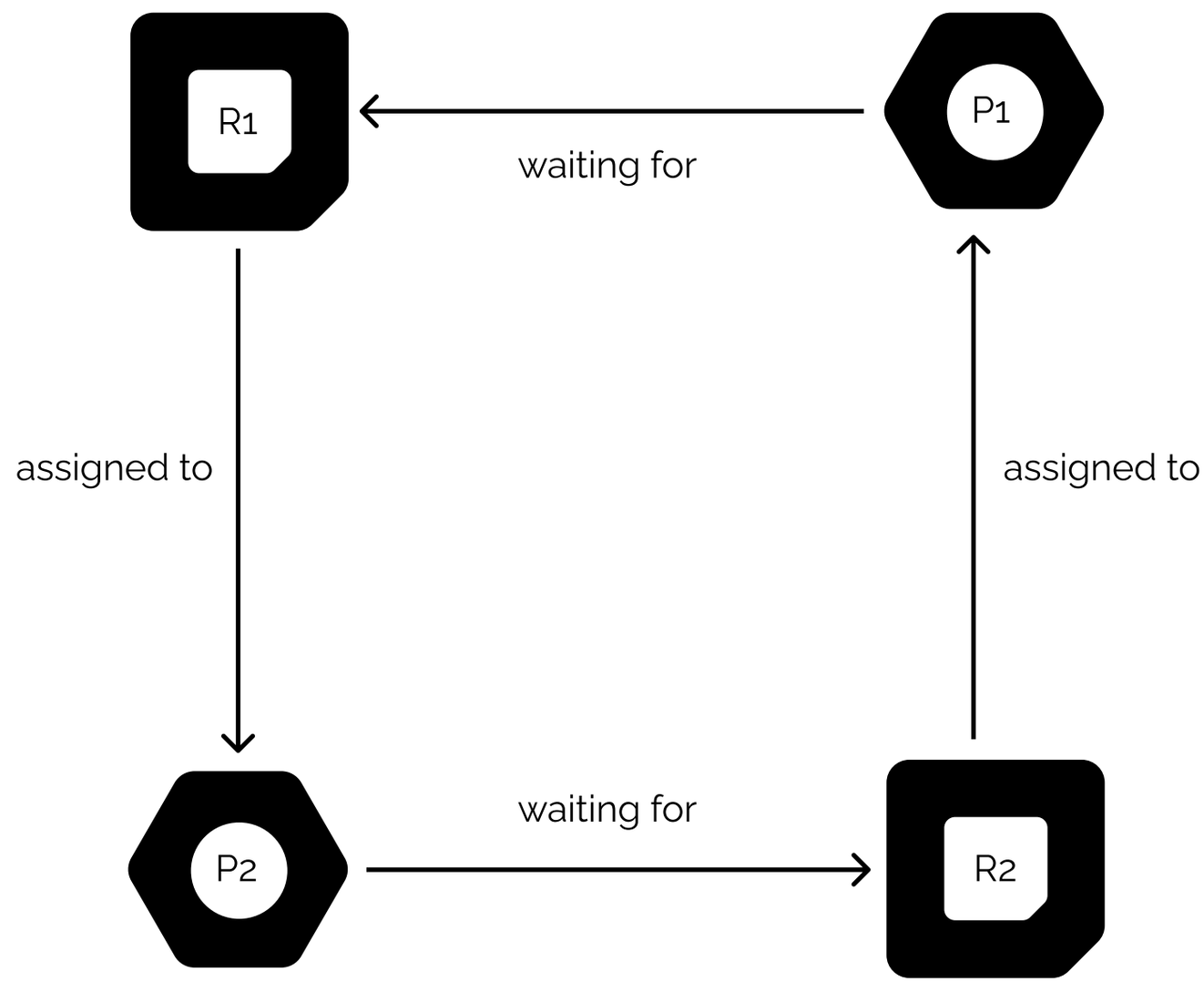
That cycle creates deadlock.

Hold and Wait

A process is holding one resource and waiting for another.

No Preemption

A resource cannot be forcibly taken away from a process.



Verdict: Deadlock

HANDLING DEADLOCK

Deadlock Prevention

Design system so at least one of the 4 conditions never happens.
Examples:

- Don't allow hold-and-wait
- Force processes to request all resources at once
- Allow preemption in some cases

Deadlock Avoidance

OS checks whether granting a resource is safe before giving it.

Most famous method: Banker's Algorithm

***Concept:** If giving a resource may lead to deadlock later, OS refuses for now.

Deadlock Detection

Let deadlock happen, then detect it.
Used when prevention/avoidance is too costly

Deadlock Recovery

After detecting deadlock, OS fixes it by:

- killing one or more processes
- taking resources back
- restarting processes

Ignore the Problem

This is surprisingly the dominant approach in general-purpose OSes.

- Called the "Ostrich Algorithm"
- Assumes deadlocks are rare and not worth the overhead

Used by:

- Linux
- macOS
- Windows

Instead, developers are responsible for writing safe multithreaded code.

Recap

The key takeaway is: deadlocks are a serious problem that can freeze processes or systems, and careful resource management — like what Banker's Algorithm teaches — helps the system stay safe and efficient.

Even if modern systems don't use the algorithm directly, the principle of checking resource safety before granting requests is still very relevant in databases, multithreading, and distributed systems.